Latent Multi-Agent Communication is Not a Problem-Specific Information Channel

Causal evidence for a task-level scaffold—and how to compress the inter-agent cache away

Sagnik Biswas
(advisor: Jiayi Tian)

Preliminary draft — July 31, 2026. Red TODOs mark experiments in flight.

Abstract

Multi-agent LLM systems increasingly let agents collaborate through *latent state*—a shared key–value (KV) cache—rather than natural language, on the assumption that this transmits richer intermediate reasoning than a text bottleneck. We test that assumption *causally*. Using a controlled intervention protocol that swaps the inter-agent cache with a *wrong-question*, *statistically synthetic*, *zeroed*, or *compressed* version across decoding budgets, we find that the final (text-emitting) agent is largely **insensitive to *problem-specific* content**: a different *same-task* question's cache helps as much as the correct one, the target answer is barely decodable from upstream latent states (probes near chance), and three independent steering methods fail to improve accuracy through the channel. What the channel provides is instead a **task-level conciseness scaffold** (not a generic free lunch across domains—a MedQA cache is not expected to serve a MATH problem). We exploit this directly: the inter-agent relay compresses **16–40× at iso-accuracy** via plain headwise eviction, or can be replaced by a fixed *same-task* synthetic cache at *zero upstream compute*; any verbosity that aggressive compression induces is removed by steering the reader. Across [TODO: N] models and [TODO: M] tasks (incl. MATH), the usable leverage in latent multi-agent systems sits at the reader, not the latent channel—with concrete implications for how such systems should be built and optimized.

1 Introduction

Multi-agent LLM pipelines improve reasoning by decomposing a task across specialized roles (e.g., plan / critique / refine) that exchange messages. A recent and appealing variant, *LatentMAS* [1], replaces the text messages with the agents' *latent* working memory: each upstream agent "thinks" by feeding its last-layer hidden state back as its next input embedding, emits no tokens, and grows a shared KV cache that the final agent reads before decoding the answer. The stated motivation is that a KV hand-off preserves richer information than a lossy text round-trip. Implicit in this design is a strong assumption: *the latent cache is an information channel that carries task-specific reasoning from the upstream agents to the reader.*

We interrogate that assumption with controlled, causal interventions on the cache and reach a more precise conclusion. The reader does not use the *problem-specific* content of the inter-agent cache: replacing it with a different *same-task* question's cache leaves accuracy essentially unchanged; the answer is not linearly (or non-linearly) decodable from upstream latents; and every attempt to *steer* the latent content—training-free and learned—fails to move accuracy. What the cache does provide is a *task-level* effect: it places the reader into a shorter, often more accurate decoding mode (a "conciseness scaffold"). We do *not* claim that any cache works—a MedQA cache on a

MATH problem is expected to hurt—and we test this specificity ladder (same-instance / within-task shuffle / cross-task / same-task synthetic) explicitly. Much of the apparent accuracy benefit of the upstream agents over a reader-only baseline is also a decoding-budget artifact.

This reframing has a direct, practical consequence. If *instance-level* content does not matter, the relay is highly redundant within a task—so it should compress drastically for free. We confirm this: the inter-agent cache compresses **16–40**× with no accuracy loss, and can even be replaced by a fixed *same-task* synthetic cache built from population statistics, eliminating upstream compute. When very aggressive compression makes the reader mildly more verbose, a reader-only concision steering vector removes exactly that overhead. The leverage is at the reader.

Contributions.

1. A **causal intervention protocol** for latent multi-agent systems (real / shuffled / zeroed / synthetic / compressed cache × decoding budget, plus per-agent readout probes and role ablations) that separates *content* effects from *presence* effects (§3).
2. Evidence that the latent inter-agent channel is **not steerable for accuracy** (three independent methods; §4) and is **content-independent**: real ≈ shuffled across models and tasks, and the answer is barely decodable upstream (§5).
3. A **practical exploit**: 16–40× inter-agent KV compression at iso-accuracy (plain headwise eviction), a zero-upstream-compute synthetic scaffold, and reader-side concision steering that cancels compression's verbosity tax (§6).

2 Background and Setup

LatentMAS. We study a verified, byte-for-byte reproduction of LatentMAS [1] with the sequential topology **Planner** → **Critic** → **Refiner** → **Judger**. Each upstream agent runs K=40 latent forward passes (last hidden state realigned and fed back as the next input embedding), emits no text, and appends to a shared, growing `past_key_values`. Only the Judger decodes text. The inter-agent "message" is therefore literally the KV cache (Fig. 1). Our fork reproduces the paper's accuracy on Qwen3-14B (GSM8K 92.7 vs. 95.2; ARC-C 94.7 vs. 95.6; MedQA 79.3 vs. 80.7).

Reader-side steering (SEAL/ASC). As one intervention we use a training-free steering vector [2]: from labeled reasoning steps we build $v = \text{mean}(\text{execution}) - \text{mean}(\text{reflection} \cup \text{transition})$ at a deep layer, and add $\alpha\, v$ to the residual stream during the Judger's decode. With $\alpha{>}0$ this suppresses reflection/transition steps, yielding shorter output. We call this reader-side concision steering (ASC).

KV compression. We compress the relay by keeping a few "sink" positions plus the top-k positions by importance per layer (headwise eviction), optionally with a low-rank back-fill of the evicted mass; see §6. This connects to KV-cache eviction methods (H2O, SnapKV) [3, 4], which we apply to the novel setting of a *multi-agent latent relay*.

Models, tasks, protocol. Primary backbone Qwen3-14B (40 layers, hidden 5120); Qwen3-4B for cross-scale checks; [TODO: Llama-3.1-8B for cross-family]. Tasks: GSM8K, MedQA, ARC-Challenge, and [TODO: MATH-500]. Unless noted, greedy decoding; accuracy and mean Judger output tokens are reported at a fixed decoding budget. Paired bootstrap 95% CIs throughout.

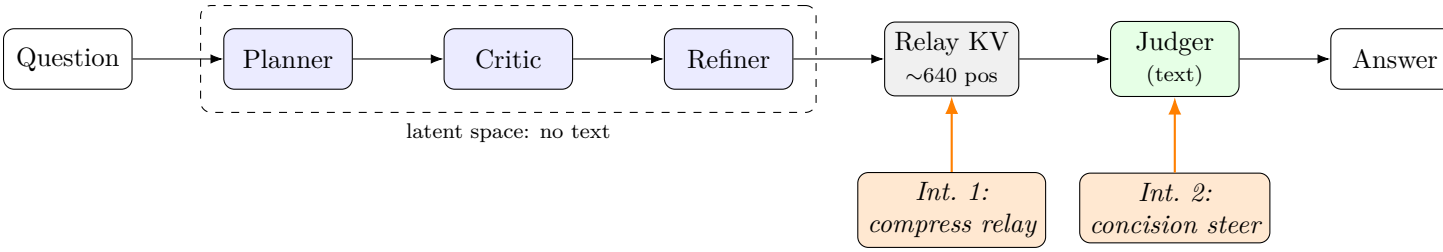

Figure 1: LatentMAS pipeline and our two interventions. Three upstream agents reason in latent space and grow a shared KV cache; only the Judger emits text. **Int. 1** compresses the relay cache (memory); **Int. 2** steers the reader toward concision (verbosity). Everything before the Judger emits no tokens.

3 A Causal Intervention Protocol for Latent MAS

Prior work establishes latent MAS *correlationally*: adding the agents raises accuracy. That is consistent with the agents transmitting useful reasoning, but equally consistent with a content-independent mode shift. To separate these, we intervene directly on the cache the reader consumes while holding the reader's prompt fixed, and vary the *decoding budget* T (max new tokens):

- **real**: the item's own upstream cache (upper reference).
- **shuffled**: another question's real cache (the placebo—same statistics and on-manifold, wrong content).
- **none**: reader-only, no cache (lower reference).
- **zeroed**: cache positions present but values set to zero (presence without content).
- **synthetic**: a cache sampled from per-channel population statistics of real caches (question-independent; zero upstream compute).
- **compressed**: the real cache after headwise eviction / low-rank back-fill (§6).

The key comparison is **real vs. shuffled**: if a wrong-question cache is as good as the right one, the reader is not using the content. We complement this with (i) per-agent, per-layer *readout probes* for the gold answer, and (ii) a *role ablation* that drops the upstream agents entirely.

4 Result 1: The Latent Channel Is Not Steerable for Accuracy

We first ask whether the channel can be *made* useful by steering its content. Three independent methods all fail (Table 1): a training-free diff-of-means direction applied to the upstream latents (within noise; hurts MedQA); one-shot KV-cache steering of the hand-off (the dense content positions saturate or corrupt—steering the aggregation position only affects efficiency, not accuracy); and a steering vector trained end-to-end through the differentiable KV (inert at the last token; *destructive-only* for all-token, collapsing to 0 at negative coefficients). No method finds an accuracy-improving direction.

5 Result 2: The Channel Is Not Problem-Specific

A specificity ladder. We distinguish three levels of content: *instance* (this problem's reasoning), *task* (same domain/format), and *none*. The claim is that the reader ignores *instance*-level content

Table 1: Steering the latent content does not improve accuracy (Qwen3-14B unless noted). Three independent methods; details in appendix.

Method	Setting	Outcome
Diff-of-means (upstream latents)	GSM8K/ARC/MedQA	within noise; hurts MedQA
KV-cache steering (hand-off)	GSM8K	falsified (content saturates/corrupts)
Learned vector (end-to-end, diff. KV)	4B/MedQA	inert or destructive-only

Table 2: Content-independence: a wrong-question cache (**shuf**) ≈ the correct one (**real**); both beat reader-only (**none**) mainly at tight budgets. Accuracy at decoding budget T.

Model	Task	T	real	shuf	none
Qwen3-14B	MedQA	1024	0.675	0.625	0.350
Qwen3-14B	MedQA	4096	0.800	0.775	0.750
Qwen3-14B	GSM8K	1024	0.933	0.933	0.800
Qwen3-4B	MedQA	1024	0.450	0.450	0.150
Qwen3-14B	MATH	2048	[TODO: running: real vs within-task shuf]		
Cross-task specificity (does domain matter?)					
Qwen3-14B	MedQA	1024	—	—	[TODO: running: donor=GSM8K]

but may still depend on *task*-level priming. Table 2 reports the core within-task result; the cross-task cell is the decisive test of whether even the domain matters.

Real ≈ within-task shuffled. Swapping in a *different same-task* question's cache costs at most a few points and usually nothing, on both MedQA and GSM8K and on both model scales. In contrast, removing the cache entirely (**none**) hurts substantially at tight budgets but the gap collapses as the budget grows—i.e., much of the "benefit" of the upstream agents is a decoding-budget artifact: given room, the reader re-derives the answer alone.

The answer is barely encoded upstream. If the cache carried the solution, we should be able to read it out. We train linear and MLP probes on each upstream agent's per-layer latent state to predict the gold answer (Table 3). They sit near chance; only at the *Judger* does a correctness signal become decodable (AUC 0.69–0.80). The decision is made late, at the reader.

Dropping the agents barely hurts. A role ablation removing the upstream agents (reader-only) matches the full pipeline within noise (GSM8K 0.91 vs. 0.93; MedQA 0.737 vs. 0.717), a third independent signal that upstream *content* carries little causal value.

Cross-model / cross-task. [TODO: multi-seed + Llama-3.1-8B + MATH to establish generality; currently single-seed on 4B/14B, GSM8K/MedQA/ARC.]

6 Result 3: So Compress It—and Steer the Reader

Content-independence predicts the relay is highly redundant. We confirm two exploits and show reader-side steering is complementary.

Table 3: Upstream latents barely encode the answer. Gold 4-way probe accuracy (chance 0.25) and correctness-probe AUC.

Agent	gold acc (MLP)	correctness AUC
Planner	0.33	0.54
Critic	0.32	0.62
Refiner	0.28	0.59
Judger	—	**0.69–0.80**

Table 4: 2×2 on GSM8K/Qwen3-14B: relay × concision steering. Compression is near-free; aggressive compression adds a verbosity tax that steering removes; plain-H eviction is the memory-optimal winner.

Relay	Steer	Acc	Tokens	Relay KV	#pos
Moderate compression (n=100, budget 768)					
Full	off	0.800	524	99.7 MB	638
Full	on	0.860	489	99.7 MB	638
Comp.	off	0.790	534	**6.25 MB**	40
Comp.	on	**0.860**	497	**6.25 MB**	40
Aggressive compression (n=60, budget 768)					
Full	off	0.817	529	100 MB	640
Full	on	0.867	488	100 MB	640
Comp. (obf)	off	0.800	567	3.75 MB	24
Comp. (obf)	on	**0.867**	523	3.75 MB	24
Evict (plain-H)	off	**0.817**	562	**2.50 MB**	16
Evict (plain-H)	on	0.800	532	**2.50 MB**	16

Compress the relay 16–40× for free. We evict all but a few "sink" + top-k positions per layer. Table 4 and Fig. 2 show a full 2×2 (relay ∈{full, compressed} × concision steering ∈{off, on}) on GSM8K/Qwen3-14B. At a moderate budget (32 kept positions, 16× smaller KV) accuracy is unchanged. Squeezing to 16–24 positions (27–40×; 100 MB→ 2.5–3.75 MB) is still within noise on accuracy, and *plain headwise eviction* (no back-fill) matches the full-cache baseline exactly (0.817) while using the least memory.

A verbosity tax, and its cure. Aggressive compression makes the reader mildly more verbose (compression tax +37 tokens, 95% CI $[+11, +67]$). Reader- side concision steering removes it (-44 tokens $[-71, -16]$) and lifts accuracy to the steered full-cache level—i.e., **memory and verbosity are separable, independently fixable knobs**. Steering delivers the same benefit on the compressed relay as on the full one (interaction ≈ 0).

Or synthesize the relay at zero upstream compute. A single fixed *synthetic* cache, sampled from per-channel statistics of a handful of real caches (question-independent), reproduces real accuracy and conciseness on 14B (MedQA, GSM8K) with no upstream forward passes—consistent with the content-independence claim. [TODO: fold synthetic-scaffold table + multi-seed.]

Whole-system accounting. Compression shrinks the reader's cache and its decode time but not the upstream latent forwards (a fixed cost paid to *build* the cache); the honest win is memory/throughput plus a modest reader latency reduction. [TODO: latency/compute/memory Pareto decomposed by stage across batch sizes.]

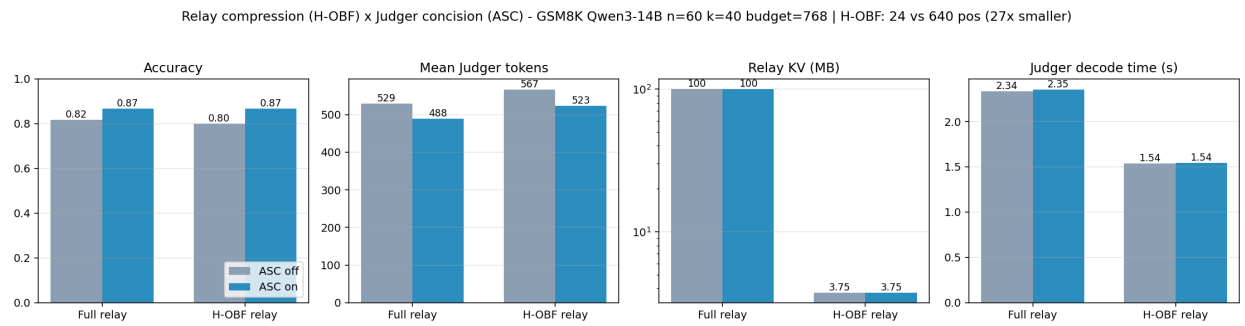

Figure 2: Aggressive compression (∼27×). Compression is near-free on accuracy; it induces a small verbosity tax that reader-side steering removes; compressed decodes are also faster. Full result in Table 4.

7 Related Work

Latent / multi-agent collaboration. LatentMAS [1] and text multi-agent systems (debate, role decomposition) [6] assume inter-agent messages carry task content; we provide a causal account showing the *latent* channel's benefit is content-independent. **Activation steering.** CAA/ITI/SEAL [2, 5] steer *token generation*; we show a non-text latent inter-agent channel is *not* steerable for reasoning quality, and that the usable surface is the reader. **KV-cache compression.** H2O, SnapKV [3, 4] evict KV for a single long context; we apply eviction to a multi-agent latent relay and explain *why* it is nearly free (content-independence).

8 Discussion and Limitations

Our claims are strongest where the reader is already capable; on harder tasks or weaker readers the agents may transmit genuine content—the [TODO: MATH] crux directly tests this and could yield a boundary condition rather than a universal claim. Grading for MATH uses a light normalizer (not full CAS equivalence). Results are currently single-seed on two model scales and 2–3 tasks; multi-seed, a second model family, and the whole-system latency Pareto are in flight. We do not claim the upstream agents are useless—only that, on the tasks tested, their benefit to the reader is not carried by the *content* of the latent relay.

9 Conclusion

The inter-agent latent cache in LatentMAS behaves as a content-independent conciseness scaffold, not an information channel: a wrong-question cache helps as much as the right one, the answer is barely decodable upstream, and the channel is not steerable for accuracy. Consequently the relay compresses 16–40× for free (or can be synthesized at zero upstream compute), and reader-side steering handles verbosity. Build and optimize latent multi-agent systems at the reader.

References

[1] Zou et al. Latent Collaboration in Multi-Agent Systems. arXiv:2511.20639, 2025.

[2] Chen et al. SEAL: Steerable Reasoning Calibration of LLMs for Free. arXiv:2504.07986, 2025.

[3] Zhang et al. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of LLMs. NeurIPS, 2023.

[4] Li et al. SnapKV: LLM Knows What You are Looking for Before Generation. NeurIPS, 2024.

[5] Rimsky et al. Steering Llama 2 via Contrastive Activation Addition. ACL, 2024.

[6] Du et al. Improving Factuality and Reasoning in Language Models through Multiagent Debate. ICML, 2024.